

第一章 第二章	3
1. 什么是数据模型, 什么是数据模式? 区别和关系是什么? (P7)	3
2. 解释数据模型的概念, 为什么要将数据模型分成两个层次?	3
3. 数据库系统包含哪三级模式 (ANSI-SPARC 架构)? 试分别说明每一级模式的作用?	3
4. 数据库三级模式划分的优点是什么? 它能带来哪些数据独立性?	3
5. 现代数据库怎么管理数据模式的? 数据模型怎么影响系统性能? 什么是结构化数据, 半结构化数据, 非结构化数据?	3
6. 数据库系统的两级映像的功能是什么? 它带来了哪些功能?	3
7. 试说明数据、数据库、数据库管理系统和数据库系统的概念以及它们之间的关系	4
8. 什么是数据库系统? 它有哪些重要组成部分? 数据模型和数据语言有什么关系?	4
9. 比较用文件管理和用数据库管理数据的主要区别	4
10. 概念数据模型和概念数据模式的区别和联系	4
11. 数据库系统由哪几部分组成, 每一部分在数据库系统中的作用大致是什么?	5
12. 现代数据库系统如何实现数据的独立性?	5
13. 数据独立性	5
14. 层次数据模型、网状数据模型、关系数据模型 (P12)	5
15. 键 (P23)	6
16. 关系代数和关系演算的概念、区别、联系	6
17. 关系演算 (元组关系演算、域关系演算) (P29)	7
18. 分布式数据库系统	8
第三章 关系数据库语言	8
1. 试说明使用视图的好处	8
2. 使用视图可以加快数据的查询速度, 这句话对吗? 为什么?	8
3. 简述 SQL 和关系代数的联系和区别	9
4. WHERE 和 HAVING 区别	9
5. 嵌入式 sql (cursor)	9
6. 画 ER 图	9
第四章 数据库管理系统引论	10
1. 试分析空值产生的原因。为了处理空值, DBMS 要做哪些主要工作?	10
2. 如事务不遵守 ACID 准则, 则对数据库产生何种后果? 为什么一般不涉及数据库的程序中不提 ACID 准则?	10
3. 何谓线程? 试分析现代 DBMS 趋向采用多线程进程结构的原因	10
4. 试分析当前客户/服务器系统结构和三层客户/服务器系统结构流行的技术背景	10
5. 试述数据目录的作用、内容及管理的特点	11
第五章 数据库的存储结构	11
1. 为什么要创建索引?	11
2. 为什么不对表中的每一个列创建一个索引?	11
3. 哪些列上可以创建索引, 在哪些列上不能创建索引?	12
4. 稠密索引是否一定能够提高针对索引属性查询的效率?	12
5. 为什么扩大数据库系统的缓冲区可以提高其性能?	12
6. 堆文件、直接文件和索引文件各适用于何种场合?	12
7. 为什么主键上一般都建有主索引?	13
8. 当查询的元组数在总元组数中所占的比重较大时, 为什么用非簇集索引有时反而不利? 为什么在此情况下用簇集索引有利?	13
9. 在数据库系统中, 散列一般用的不如索引多, 试分析其原因。	13
10. 如何减少 B-树的级数?	13
11. B 树和 B+树有什么区别? 为什么在 DBMS 中使用 B+树? (P107)	14
12. B-树为代表的树形索引成为当前数据库系统主流索引是否具有必然性?	14

13. 计算 B+树的秩 k	14
第六章 查询处理优化	14
1. 从查询优化的角度解释为什么在 SQL 的 WHERE 子句中应该避免使用“OR” (P124)	14
2. 数据模型的优化包含哪些方法?	15
3. 判断每个关系模式的范式, 根据实际需要确定最合适的范式。	15
3. Join 的优化	15
4. 单纯代数优化的局限性	15
第七章 事务管理	15
1. 数据冗余的优缺点是什么?	15
2. 数据库恢复的基本原则是什么? 具体实现方法是什么? (P138)	15
3. 列出 DBMS 三种恢复策略, 并描述提交规则和提前记录规则对于更新事务的必要性。	16
4. 失效及恢复对策 (P146)	16
5. 登记日志文件时为什么必须先写日志文件, 后写数据库?	16
6. 试述使用检查点方法进行恢复的步骤	16
7. Checkpoint	16
8. 试叙述事务的 4 个性质, 并结合每一个性质由 DBMS 的哪个子系统实现? 每一个性质对 DBS 有什么益处?	17
9. 什么是数据库镜像? 它有什么用途?	17
10. 在数据库中为什么要有并发控制机制?	17
11. 锁 (P153)	17
12. 各种数据有不同的恢复要求: 有些要求恢复至最近一致状态; 有些只要求恢复至先前的某个一致状态; 也有些数据不需要恢复措施, 只需在失效后重建, 试各举一例	19
13. 试论述提交规则和先记后写规则对更新事务的必要性	19
14. 说明 redo、undo 操作内容及其所需的条件, 在什么情况下需要 redo, 在什么情况下需要 undo?	19
15. 讨论异地更新恢复技术的优缺点。在异地更新方案中, 是否需要运行记录? 在什么环境下可以省去运行记录?	20
16. 什么叫并发? 为什么要并发运行? 并发会引起什么问题? 什么样的并发执行才是正确的? 如何避免并发所引起的问题?	20
第八章 数据库安全和完整性约束	21
1. 什么是数据库的完整性? 它和数据库的安全性有什么不同?	21
2. 关系模式的设计考虑哪些问题?	21
第九章 触发器和主动数据库系统	21
1. 解释什么是触发器并列出发器的组件	21
2. 试编写一个触发器 (P185)	22
FOR EACH STATEMENT	22
第十章 数据依赖和关系模式的规范化	22
1. 是否存在“删除异常”和“修改异常”问题? 如果存在, 请说明分别是怎样发生的? (P198)	22
2. 数据模式遵循的范式越高越好吗?	22
3. 范式	22
SQL	23

第一章 第二章

1. 什么是数据模型，什么是数据模式？区别和关系是什么？（P7）

数据模型是用于描述数据的概念和定义的集合， 比如字段

数据模式是使用给定数据模型对特定数据集合的描述，比如学生信息记录可以定义为姓名、学号、性别等属性和关系的形式

区别：数据模型是描述数据的手段，而数据模式是用给定的数据模型对具体的数据的描述

关系：数据模式反映一个单位的各种事物的结构、属性、联系和约束，实质上是用数据模型对一个单位的模拟

2. 解释数据模型的概念，为什么要将数据模型分成两个层次？

数据模型是对现实世界数据特征的抽象。数据模型一般要满足三个条件：第一是数据模型要能够比较真实地模拟现实世界；第二是数据模型要容易被人们理解；第三是数据模型要能够很方便地在计算机上实现。由于用一种模型同时很好地满足这三方面的要求在目前是比较困难的，因此在数据库系统中就可以针对不同的使用对象和应用目的，采用不同的数据模型。根据模型应用的不同目的，将这些模型分为概念层数据模型和组织层数据模型两大类，以方便对信息的描述。

3. 数据库系统包含哪三级模式（ANSI-SPARC 架构）？试分别说明每一级模式的作用？

数据库系统包含的三级模式为：内模式、模式和外模式。外模式是对现实系统中用户感兴趣的整体数据结构的局部描述，用于满足不同数据库用户需求的数据视图，是数据库用户能够看见和使用的局部数据的逻辑结构和特征的描述，是对数据库整体数据结构的子集或局部重构。模式是数据库中全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图。内模式是对整个数据库的底层表示，它描述了数据的存储结构。

4. 数据库三级模式划分的优点是什么？它能带来哪些数据独立性？

数据库三级模式的划分实际上将用户、逻辑数据库与物理数据库进行了划分，使彼此之间相互干扰减到最少。这三个模式的划分实际带来两个数据独立性，即物理独立性和逻辑独立性。这使得底层的修改和变化尽量不影响到上层。

5. 现代数据库怎么管理数据模式的？数据模型怎么影响系统性能？什么是结构化数据，半结构化数据，非结构化数据？

数据库系统划分为三个层次，称为三级模式，分别为概念模式，外模式，内模式，都存于数据目录中，是数据目录的最基本内容，DBMS 通过数据目录，管理和访问数据模式。

数据模型包含数据结构，数据操作，数据完整性约束，影响性能的因素主要是数据结构的复杂度和数据操作的可优化程度。

结构化数据：数据整体结构化， 通过数据模型描述

半结构化数据：单条记录内部的数据有结构，数据文件间无联系，整体无结构

非结构化数据：数据间，数据文件间都没有结构

6. 数据库系统的两级映像的功能是什么？它带来了哪些功能？

数据库系统的两级映像是外模式 / 模式映像和模式 / 内模式映像。模式 / 内模式映像定义了概念视图和存储的数据库的对应关系，它说明了概念层的记录和字段在内部层次怎样表示。如果数据库的存储

结构改变了，那么，必须对模式 / 内模式映像进行必要的调整，使模式能够保持不变。外模式 / 模式映像定义了特定的外部视图和概念视图之间的对应关系，当概念模式的结构可发生改变时，也可以通过调整外模式 / 模式映像间的映像关系，使外模式可以保持不变。

7. 试说明数据、数据库、数据库管理系统和数据库系统的概念以及它们之间的关系

(1) 数据(Data)：描述事物的符号记录称为数据。数据的种类有数字、文字、图形、图像、声音、正文等。数据与其语义是不可分的。

(2) 数据库(Database, 简称 DB)：数据库是长期储存在计算机内的、有组织的、可共享的数据集合。数据库中的数据按一定的数据模型组织、描述和储存，具有较小的冗余度、较高的数据独立性和易扩展性，并可为各种用户共享。

(3) 数据库系统(Database System, 简称 DBS)：数据库系统是指在计算机系统中引入数据库后的系统构成，一般由数据库、数据库管理系统(及其开发工具)、应用系统、数据库管理员构成。

(4) 数据库管理系统(Database Management System, 简称 DBMS)：数据库管理系统是位于用户与操作系统之间的一层数据管理软件，用于科学地组织和存储数据、高效地获取和维护数据。DBMS 主要功能包括数据库的建立和维护功能、数据定义功能、数据组织存储和管理功能、数据操作功能、事务的管理和运行功能。

它们之间的联系：数据库系统包括数据库、数据库管理系统、应用系统、数据库管理员，所以数据库系统是个大的概念。数据库是长期存储在计算机内的有组织、可共享的大量的数据集合，数据库管理系统是由管理员操作管理数据库的查询、更新、删除等操作的，数据库应用系统是用来操作数据库的。

8. 什么是数据库系统？它有哪些重要组成部分？数据模型和数据语言有什么关系？

(1) 数据库系统面向数据密集型应用，以统一管理和共享数据为特征，具有存储、管理、维护数据的功能。

(2) 数据库系统由应用程序、数据库管理系统、数据库、数据库管理员等组成。

(3) 数据模型是用来描述现实世界数据的一组概念和定义；数据语言（数据库语言）包括数据定义、操作、查询等基本组成部分（可以以关系模型和 SQL 为例进一步展开）。

9. 比较用文件管理和用数据库管理数据的主要区别

文件系统把数据组织成相互独立的数据文件，实现了记录内的结构性，但整体无结构；而数据库系统实现整体数据的结构化，这是数据库的主要特征之一，也是数据库系统与文件系统的本质区别。

在文件系统中，数据冗余度大，浪费存储空间，容易造成数据的不一致；数据库系统中，数据是面向整个系统，数据可以被多个用户、多个应用共享使用，减少了数据冗余。

文件系统中的文件是为某一特定应用服务的，当要修改数据的逻辑结构时，必须修改应用程序，修改文件结构的定义，数据和程序之间缺乏独立性；数据库系统中，通过 DBMS 的两级映像实现了数据的物理独立性和逻辑独立性，把数据的定义从程序中分离出去，减少了应用程序的维护和修改。

文件系统和数据库系统均可以长期保存数据，由数据管理软件管理数据，数据库系统是在文件系统基础发展而来。

数据库系统与文件系统相比实际上是在应用程序和存储数据的数据库之间增加了一个系统软件，即数据库管理系统，使得以前在应用程序中由开发人员实现的很多繁琐的操作和功能，都可以由这个系统软件完成，这样应用程序不再需要关心数据的存储方式，而且数据的存储方式的变化也不再影响应用程序。而在文件系统中，应用程序和数据的存储是紧密相关的，数据的存储方式的任何变化都会影响到应用程序，因此不利于应用程序的维护。

10. 概念数据模型和概念数据模式的区别和联系

概念数据模型是对数据的高层次抽象，描述核心概念和关系，关注业务需求，独立于技术细节，常用 ER 图表示，面向用户需求沟通，为逻辑和物理模型设计奠定基础。

概念数据模式是概念数据模型的具体实现，描述数据库逻辑结构，是数据库设计蓝图，接近实现层次，独立于物理存储但与逻辑模型相关，用于定义表结构和约束，适合设计人员使用。

两者区别在于模型关注业务抽象，模式更接近逻辑设计，模型独立于技术，模式面向实现，二者在数据库设计中层层递进，密切衔接。

联系：

设计衔接：概念数据模型是捕获用户需求的高层抽象，概念数据模式基于其进行逻辑结构的具体化。

抽象到实现：概念数据模型提供业务语义，概念数据模式将其转化为数据库的逻辑设计。

目标一致：都旨在设计满足业务需求的数据库系统。

内容延续：模型中的实体、属性和关系在模式中具体化为表、字段和约束。

依赖关系：数据模型是模式的基础，模式为后续的物理实现提供依据。

两者是递进关系，概念数据模型为模式设计奠定基础，模式则实现模型的逻辑表达。

11. 数据库系统由哪几部分组成，每一部分在数据库系统中的作用大致是什么？

数据库系统由三个主要部分组成，即数据库、数据库管理系统和应用程序。数据库是数据的汇集，它以一定的组织形式存于存储介质上；数据库管理系统是管理数据库的系统软件，它可以实现数据库系统的各种功能；应用程序指以数据库数据为核心的应用程序。

12. 现代数据库系统如何实现数据的独立性？

两级映射和三级结构

13. 数据独立性

数据独立性越来越高，为什么数据管理系统的数据独立性越高越好？

数据独立性是指建立在数据的逻辑结构和物理结构分离的基础上，用户只需要以简单的逻辑结构操作数据而无需考虑数据实际的物理结构，中间的转换工作由数据库管理系统实现。数据独立性分为数据的物理独立性和数据的逻辑独立性。

✓ 数据的物理独立性

➤ 含义：是指用户的应用程序与存储在磁盘上的数据库中数据是相互独立的。用户程序不需要了解实际的物理结构，只要处理数据的逻辑结构即可。

➤ 意义：

[1] 可以实现数据的物理存取与应用程序实现相分离。

[2] 当数据的物理存储结构与存取方法的改变时，不要求修改应用程序。

[3] 使初步数据共享成为可能，只要知道数据存取结构，不同程序可共用同一数据文件。

✓ 数据的逻辑独立性

➤ 含义：用户的应用程序与数据库的逻辑结构是相互独立的，即当数据的逻辑结构改变时，用户程序也可以不变。

➤ 意义：将数据的使用与数据的逻辑结构相分离

14. 层次数据模型、网状数据模型、关系数据模型（P12）

14.1 与网状、层次模型的数据库管理系统相比，RDBMS 的查询处理有什么本质的不同？

本质不同：关系数据库系统采用非过程化的查询语言

14.2 与层次结构和网络数据库系统相比，查询优化对于关系数据库系统更为重要，是吗？为什么？

正确。因为层次和网状数据库的查询语言一般是过程语言，层次结构和网状结构本身就结构较为简单，很多都不包含属性，是按树或者链的型式表示的，在查询时只是沿链做简单的查询。但关系型数据库系统的关系模型是基于关系(表)组织的，在查询时会涉及很多拼接两个表的操作，这个操作是很耗时的，

所以在查询时进行优化减小拼接表的工作量和时间非常重要。关系数据库的查询语言一般是非过程语言，仅表达查询的要求，而不说明查询的过程，用户不关心查询语言的具体执行过程，由 DBMS 确定合理有效的执行策略，查询优化是查询处理中重要必要的一环，对系统的性能影响颇大，关系数据库比层次和网状数据库复杂，因此对其来说查询优化更加重要。

14.3 在网状数据模型和关系数据模型中，如何表达两个记录型之间的 $m:n$ 关系？一般而言，三个实体型之间的多对多联系和这三个实体型两两之间的三个多对多联系等价吗？分别举例说明。

(1)网状数据模型中，通过 Link 记录来实现 $M:N$ 关系，而关系模型中通过关系（表）来表示 $M:N$ 关系，与网状数据模型相比，实现的是一种“软连接”；

(2)不等价；

(3)以供应商、零件、工程三者的关系为例，他们之间的一个三元多对多关系和三个两两之间的二元多对多关系是不同的，因为如果供应商 A 能够提供零件 B、A 又是工程 C 的供应商、而工程 C 又需要零件 B，并不意味着工程 C 中必须要使用供应商 A 提供的零件 B，也就是说三个两两之间的二元关系并不能决定他们之间的一个三元关系。

14.4 在网状数据模型和关系数据模型中，如何表达两个记录型之间的 $m:n$ 关系？关系模型的优点和存在的问题是什么？你认为是否会有另外一种数据模型像当初关系模型取代层次、网状模型那样取代关系模型而成为主流数据模型？为什么？

(1)网状数据模型中，通过 Link 记录来实现 $M:N$ 关系，而关系模型中通过关系（表）来表示 $M:N$ 关系；

(2) 优点：关系模型中将现实世界中的实体以及实体之间的联系统一用关系（表）来表示，与层次和网状数据模型中用物理指针表示实体间的联系相比，关系模型实现的是一种“软连接”，由于被查询的对像是关系，查询结果还是关系，因此可构成一个代数系统，即关系代数。这使得我们可以用数学的方法来研究数据库中的问题，加之关系模型概念非常简单，又解决了查询效率的问题，因此自 70 年代出现后很快就取代层次和网状数据模型而成为主流；

缺点：以记录为基础，不能很好面向用户和应用；不能以自然的方式表达实体间联系；语义贫乏；数据类型太少，不能满足应用需要；

(3) 关系模型至今仍是主流，是否会有新的数据模型取代它成为主流可根据自己的理解讨论，这是一个开放的问题，只要分析合理即可。

15. 键 (P23)

- 候选键(candidate key)：某一属性或属性组的值唯一决定其他所有属性的值，而其任何真子集无此性质
- 超键(super key)：属性组能决定其他属性的值，且其真子集也具有该性质
- 主键(primary key)：从候选键中选择一个为主键
- 候补键(alternate key)：除了主键的候选键
- 全键(all key)：主键由所有属性构成

16. 关系代数和关系演算的概念、区别、联系

relational algebra: 需要保证运算顺序，即对查询过程给出确定性描述，procedural。

relational calculus: 不需要知道运算过程和运算符顺序，non-procedural。

关系代数是一种过程式查询语言，要求明确指定查询操作的顺序和过程。它通过一系列运算符来操作数据库中的关系（表），例如选择、投影、联合、差集、笛卡尔积等。关系代数关注的是查询的执行步骤，用户需要描述查询的运算顺序，系统按照指定的顺序执行查询操作。

关系演算是一种非过程式查询语言，用户只需要描述查询的目标和条件，而不需要关心具体的查询过程

或运算顺序。关系演算通过条件表达式来定义查询，通常表现为一种“我需要满足某些条件的结果”的描述。它可以通过元组关系演算（TRC）或域关系演算（DRC）来表示查询条件。

区别在于，关系代数是过程式的，它关注查询的执行顺序和步骤，需要明确指定如何操作数据；而关系演算是非过程式的，它只关注查询的逻辑和结果，不需要指定操作过程或顺序。

联系在于，两者都是关系数据库查询语言，用于描述如何从关系数据库中检索数据。它们都可以完成类似的查询任务，并且关系演算和关系代数在理论上是等价的，即任何使用关系代数描述的查询都可以使用关系演算描述，反之亦然。两者的主要不同之处在于查询的描述方式：关系代数偏向于操作顺序和过程，而关系演算则更关注结果和条件。

17. 关系演算（元组关系演算、域关系演算）（P29）

17.1 关系代数操作转为域关系演算和元组关系演算

<u>sid</u>	<u>sname</u>	rating	age
------------	--------------	--------	-----

S1、S2

bid	<u>bname</u>	color
-----	--------------	-------

B1

<u>sid</u>	bid	day
------------	-----	-----

R1

Conver the query of $\pi_{sid,sname,age}(\sigma_{rating < 10}(S_2))$ to DRC(域关系演算) formula and TRC(元组关系演算) formula

DRC: $\{(sid, sname, age) | (sid, sname, age) \in S_2 \wedge rating < 10\}$

TRC: $\{t[sid, sname, age] | t \in S_2 \wedge rating < 10\}$

Conver the query of $\pi_{sname, rating}(\sigma_{S_1.sid > R_1.sid}(S_1 \times R_1))$ to DRC(域关系演算) formula and TRC(元组关系演算) formula

DRC:

$$\left\{ \langle sname, rating \rangle | \exists S_1.sid, sname, rating, age, R_1.sid, bid, day \right. \\ \left. \left((S_1.sid, sname, rating, age) \in S_1 \wedge (R_1.sid, bid, day) \in R_1 \wedge (S_1.sid > R_1.sid) \right) \right\}$$

TRC:

$\{t[sid, sname, age] | t = \langle t^s, t^r \rangle \wedge t^s \in S_1 \wedge t^r \in R_1 \wedge (t[S_1.sid] > t[R_1.sid])\}$

使用域关系演算来表达除法运算

设有关系 R (X, Y) 和 S (Y)，X 和 Y 为属性组。则域关系演算表达式： $\{X | \exists X \forall Y (R(X, Y) \text{ AND } S(Y))\}$

17.2 元组演算

$R=(A,B,C)$ $S=(D,E,F)$ 设关系 $r(R)$ 和 $s(S)$ 已知, 分别给出与下列表达式等价的元组关系演算表达式:

- (1) $\Pi_A(r) = \{t(A) \mid t \in r\}$
 - (2) $\sigma_{B=17}(r) = \{t \mid t \in r \wedge t.B=17\}$
 - (3) $r \times s = \{t[ABCDEF] \mid t[ABC] \in r \wedge t[DEF] \in s\}$
 - (4) $\Pi_{A,F}(\sigma_{C=D}(r \times s)) = \{t[AF] \mid t[ABC] \in r \wedge t[DEF] \in s \wedge t.C=t.D\}$
2. (1) $\delta_{B=17}(r) = \{t \mid t \in r \wedge t.B=17\}$
(2) $\Pi_{A,F}(\delta_{C=D}(r \times s)) = \{t[AF] \mid t[ABC] \in r \wedge t[DEF] \in s \wedge t.C=t.D\}$
(3) $r *_{C=D} s = \{t[ABCDEF] \mid (t[ABC] \in r \wedge t[DEF] \in s \wedge t.C=t.D) \vee (t[ABC] \in r \wedge \neg(t.C \in s[D]) \wedge t.D=NULL \wedge t.E=NULL \wedge t.F=NULL)\}$

17.3 关系数据模型上的关系操作是什么? 它们的区别和相似之处是什么?

关系演算和关系代数

不同: 关系代数需要指定运算顺序; 而关系演算只需要指示逻辑条件, 结果必须满足。

相似: 它们在表达上是等价的。SQL 语言可以表达任何可以用关系代数/微积分表达的查询。

18. 分布式数据库系统

试分析分布式数据库系统出现的技术背景和应用背景。它与后来出现的联邦式数据库系统的类似之处和本质区别是什么? 为什么虽然主流的数据库产品基本上都支持分布式数据库功能, 但实际应用中分布式数据库的成功案例并不多?

背景: 分布式处理技术的发展, 当时网络宽还不足, 因此出现了由网络连接的多台计算机共同协作解决大量数据的存储、管理、查询的需求, 通过将数据就近存放提高访问效率。

本质区别: 前者在物理上分布的, 但逻辑上却是集中的。这种分布式数据库只适宜用途比较单一的、不大的单位或部门。而联邦式分布数据库系统在物理上和逻辑上都是分布的。由于组成联邦的各个子数据库系统是相对“自治”的, 这种系统可以容纳多种不同用途的、差异较大的数据库, 比较适宜于大范围内数据库的集成。

相似之处: 分布式数据库系统的不同类别。是在集中式数据库系统的基础上发展来的。是数据库技术与网络技术结合的产物。包含分布式数据库管理系统(DDBMS)和分布式数据库(DDB)。

应用并不理想的原因: ①系统实现和使用都比较复杂, 从数据库设计角度看, 物理上分布、逻辑上集中的数据库设计对开发人员要求较高; ②难于管理, 单个 DBA 很难维护大量物理上分布的数据库, 多个 DBA 又很难协调, 系统安全较难控制; ③网络带宽和服务器性能的飞速提升, 使当初的一些瓶颈不再成为障碍, 基于集中式数据库服务器的三层(多层)信息系统架构成为流行。

第三章 关系数据库语言

1. 试说明使用视图的好处

- 1、简化数据查询语句
- 2、使用户能从多角度看待同一数据
- 3、提高了数据的安全性
- 4、提供了一定程度的逻辑独立性

2. 使用视图可以加快数据的查询速度, 这句话对吗? 为什么?

不对。因为数据库管理系统在对视图进行查询时, 首先检查要查询的视图是否存在, 如果存在, 则从数据字典中提取视图的定义, 把视图的定义语句对视图语句结合起来, 转换成等价的对基本表的查询, 然后再执行转换后的查询, 所以使用视图不可以加快数据的查询速度。

3. 简述 SQL 和关系代数的联系和区别

关系代数（以关系代数为基础的数据库语言是过程性的）、SQL、关系演算（元组关系演算和域关系演算），它们的非过程化程度依次递增，主要应用领域也不同。SQL 是关系数据库的标准语言，关系代数和关系演算是它的理论基础

联系：关系代数是 sql 的理论基础。

区别：sql 是结构化查询语言，是数据库具体的技术标准和规范。关系代数是数学理论。

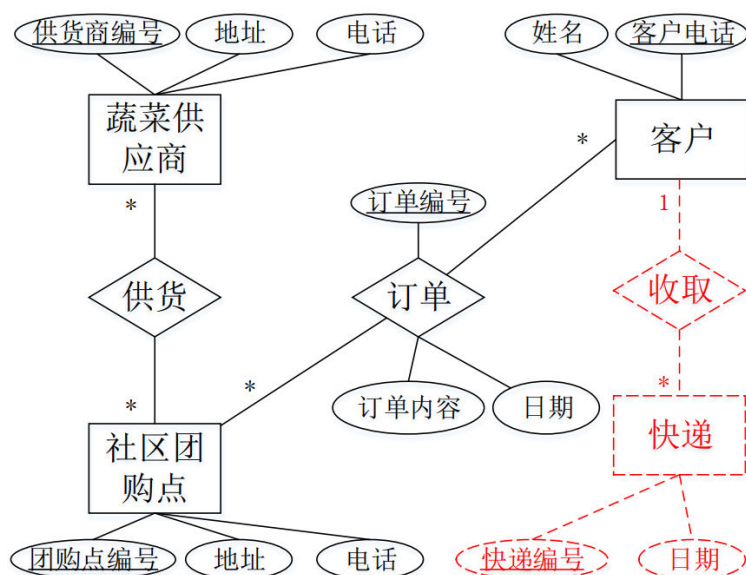
4. WHERE 和 HAVING 区别

WHERE 子句与 HAVING 短语的根本区别在于作用对象不同。WHERE 子句作用于基本表或视图，从中选择满足条件的元组。HAVING 短语作用于组，从中选择满足条件的组。

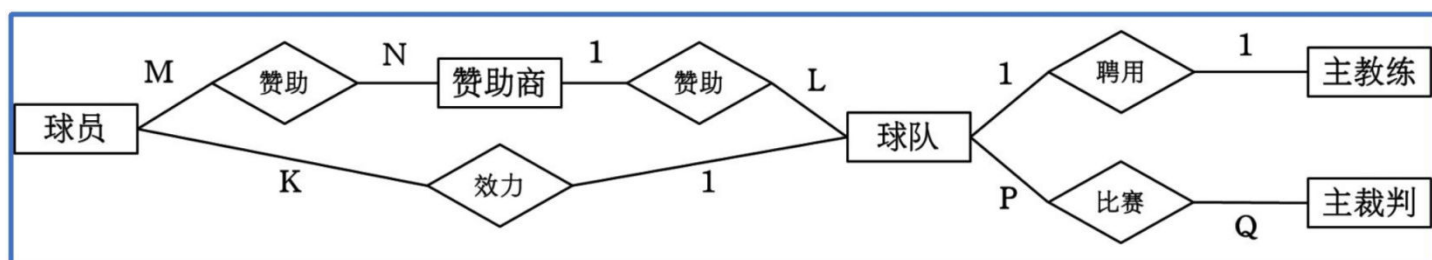
5. 嵌入式 sql (cursor)

需要有定义游标，打开游标，取数，关闭游标的语句

```
// 定义宿主变量
EXEC SQL BEGIN DECLARE SECTION;
float RATING;
int NUM;
EXEC SQL END DECLARE SECTION; // 连接数据库
EXEC SQL CONNECT :uid IDENTIFIED BY :pwd; // 使用游标读取数据
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT rating, COUNT(sid)
    FROM Sailors
    GROUP BY rating
EXEC SQL OPEN C1;
while (TRUE)
{
    EXEC SQL FETCH C1 INTO :RATING, :NUM;
    if (SQLCA.SQLCODE == 10)
        break;
    if (SQLCA.SQLCODE < 0)
        break;
    (循环打印数据) .....
}
EXEC SQL CLOSE C1; //关闭游标
```



6. 画 ER 图



第四章 数据库管理系统引论

1. 试分析空值产生的原因。为了处理空值，DBMS 要做哪些主要工作？

(1) 原因：①某些数据不确定；②某些数据当前仍不知道；③某些数据不存在；
(2) DBMS 需要在查询处理中支持包括 True、False、Null 的三值逻辑；需要支持针对空值的实体完整性约束、引用完整性约束等。

2. 如事务不遵守 ACID 准则，则对数据库产生何种后果？为什么一般不涉及数据库的程序中不提 ACID 准则？

若事务不遵守 ACID 准则，数据库中会产生脏数据，数据不一致、难以恢复等情况。不涉及数据库的程序不提 ACID 是因为它们很少需要满足数据库这样的高并发性和一致性需求。

一个事务是由应用程序中对数据库的一组操作序列组成的。如果事务不遵守 ACID 准则，则数据库中数据的完整性和一致性等就可能会因为事务的执行而遭到破坏。而一般不涉及数据库的程序不存在多用户之间数据的共享问题，所以在一般不涉及数据库的程序中不提 ACID 准则。

3. 何谓线程？试分析现代 DBMS 趋向采用多线程进程结构的原因

线程是程序执行的基本单元，是操作系统能够独立调度和分配的最小执行单元。线程是进程的一部分，一个进程可以包含多个线程，线程共享进程的资源（如内存空间、文件描述符等），但每个线程有自己的运行栈和程序计数器。

原因：

提高并发性能：多线程允许多个任务同时执行，在多核处理器上能够充分利用硬件资源，从而提高数据库的并发处理能力。

资源共享：线程共享进程的资源（如内存、缓存），线程之间的通信成本远低于进程之间的通信成本（如通过消息队列或管道），这使得多线程的 DBMS 在性能上优于多进程结构。

降低上下文切换开销：线程切换的成本比进程切换低得多，减少了 CPU 时间的浪费，提升了系统的响应速度和吞吐量。

简化任务管理：多线程使得任务分配和管理更加灵活，比如处理用户查询、日志写入、缓存管理等都可以分配到不同的线程，而不需要创建额外的进程。

更高的伸缩性：多线程模型在负载增加时更易于扩展，DBMS 可以动态创建或销毁线程来处理任务，从而更高效地利用系统资源。

异步和并行任务处理：多线程允许 DBMS 同时处理多个查询请求、后台任务（如索引重建、数据备份）和系统维护任务，提升系统整体效率。

适应现代硬件架构：现代硬件（如多核 CPU）支持线程级并行处理，多线程模型能够更好地发挥硬件性能，尤其是在大规模并发请求的场景下。

易于实现复杂功能：多线程模型更容易实现异步 I/O 操作、并行查询优化等复杂功能，这些功能对现代高性能 DBMS 至关重要。

4. 试分析当前客户/服务器系统结构和三层客户/服务器系统结构流行的技术背景

当前客户/服务器系统结构流行的技术背景包括以下方面：硬件成本降低和计算能力的提升，使客户端设备可以承担部分计算任务；网络通信技术的发展，如局域网和互联网的普及，为客户端与服务器之间的快速通信提供了支持；数据库技术的成熟，使服务器能够集中管理和处理数据，而客户端负责数据展示和交互；操作系统支持多任务和多用户，为 C/S 架构提供了稳定的运行环境；随着业务需求的复杂化，多用户并发操作需求增加，C/S 架构通过分布式计算满足了这一需求；客户端的图形用户界面技术发展，使 C/S 架构成为实现用户友好界面的首选。

三层客户/服务器系统结构流行的技术背景包括以下方面：随着业务复杂性增加，传统两层结构难以应对复杂逻辑和快速变化的需求，三层结构将应用逻辑从客户端和服务器分离出来，提高了灵活性和可维护性；分布式计算和中间件技术的发展使增加中间层成为可能，方便了业务逻辑的集中管理；互联网的普及推动了基于浏览器的轻量级客户端的兴起，而三层架构降低了客户端的开发和维护成本；三层架构支持模块化开发，促进了软件复用；技术标准化（如 Java EE、.NET 等）的普及使三层架构实现更加便捷；三层架构支持负载均衡和集群部署，提升了系统扩展性和并发能力；三层架构通过中间层对客户端和数据库之间的访问进行隔离，提高了安全性。

5. 试述数据目录的作用、内容及管理的特点

数据目录的作用

提供数据的全局描述：数据目录记录了数据库中所有对象的结构和属性信息，为数据库的访问和操作提供基础。

支持数据库管理：通过数据目录，DBMS 能够执行数据的定义、存储、访问控制以及优化查询等操作。

提供数据独立性：通过数据目录，数据库实现了物理存储和逻辑结构的分离，支持数据的物理独立性和逻辑独立性。

提供系统信息：数据目录包含系统信息，例如表空间、索引、视图等，支持 DBMS 的运行与维护。

支持多用户访问：数据目录存储了用户权限和安全信息，为用户访问数据库提供权限控制。

数据目录的内容

数据结构信息：如表、视图、索引、约束等的定义，包括字段名、数据类型、主键、外键等。

存储信息：如表空间、数据文件的存储位置和分配方式。

权限和安全信息：如用户权限、角色定义和访问控制列表。

性能和统计信息：如表和索引的统计信息，用于优化查询性能。

系统配置信息：如系统参数设置、数据库版本信息等。

日志和备份信息：如事务日志的存储位置和数据库备份记录。

数据目录管理的特点

自动管理：数据目录由 DBMS 自动维护，用户无需直接操作。

一致性和完整性：数据目录的数据必须保持一致，任何对数据库结构的修改都会实时反映到数据目录中。

高效性：数据目录的访问必须高效，以支持 DBMS 对元数据的频繁查询。

安全性：数据目录包含敏感的元数据，必须受到严格的权限控制，防止未授权访问。

动态性：随着数据库的操作，数据目录会动态更新，如新增表、修改字段等。

系统关键性：数据目录是 DBMS 运行的核心组件，其损坏可能导致数据库系统无法正常运行。

第五章 数据库的存储结构

1. 为什么要创建索引？

因为创建索引可以大大提高系统的性能。

第一，通过创建唯一性索引，可以保证数据库表中每一行数据的唯一性。

第二，可以大大加快数据的检索速度，这也是创建索引的最主要的原因。

第三，可以加速表与表之间的连接，特别是在实现数据的参考完整性方面特别有意义。

第四，在使用分组和排序子句进行数据检索时，同样可以显著减少查询中分组和排序的时间。

第五，通过使用索引，可以在查询的过程中，使用优化隐藏器，提高系统的性能。

2. 为什么不对表中的每一个列创建一个索引？

第一，创建索引和维护索引要耗费时间，这种时间随着数据量的增加而增加。

第二，索引需要占物理空间，除了数据表占数据空间之外，每一个索引还要占一定的物理空间，如果要

建立聚簇索引，那么需要的空间就会更大。

第三，当对表中的数据进行增加、删除和修改的时候，索引也要动态的维护，这样就降低了数据的维护速度。

3. 哪些列上可以创建索引，在哪些列上不能创建索引？

应该在哪些列上创建索引：

- 1、在经常需要搜索的列上建立索引可以加快检索速度；
- 2、在作为主键的列上，强制该列的唯一性和组织表中数据的排列结构；
- 3、在经常用在连接的列上，这些列主要是一些外键，可以加快连接的速度；
- 4、在经常需要根据范围进行搜索的列上创建索引，因为索引已经排序，其指定的范围是连续的；
- 5、在经常需要排序的列上创建索引，因为索引已经排序，这样查询可以利用索引的排序，加快排序查询时间；
- 6、在经常使用在 WHERE 子句中的列上创建索引，加快条件的判断速度。

不应该创建索引的的这些列具有下列特点：

1. 对于那些在查询中很少使用或参考的列不应创建索引。这是因为既然这些列很少使用到，因此有索引或者无索引并不能提高查询速度。相反，由于增加索引反而降低了系统的维护速度和增大了空间需求。
2. 对于那些只有很少数据值的列也不应增加索引。这是因为由于这些列的取值很少，例如人事表的性别列，在查询的结果中，结果集的数据行占了表中数据行的很大比例，即需要在表中搜索的数据行的比例很大。增加索引并不能明显加快检索速度。
3. 对于那些定义为 text, image 和 bit 数据类型的列不应增加索引。因为这些列的数据量要么相当大，要么取值很少。
4. 当修改性能远远大于检索性能时，不应创建索引。这是因为修改性能和检索性能是互相矛盾的。当增加索引时，会提高检索性能但是会降低修改性能。当减少索引时，会提高修改性能但会降低检索性能。因此，当修改性能远远大于检索性能时，不应创建索引。

4. 稠密索引是否一定能够提高针对索引属性查询的效率？

不一定。

- ① 如果是查询小文件中全部或相当多的记录时，使用索引并不能提高查询效率，反而会因为索引查询增加开销；
- ② 如果稠密索引为次索引，且不是簇集索引，则再最低索引中，每个键值对应的不是一个地址而是一个地址集。很可能一个键值对应的多条记录分散在不同的物理块中；当一个键值对应的记录较多时，取这些记录时访问物理块的 I/O 开销反而会降低查询的效率。

5. 为什么扩大数据库系统的缓冲区可以提高其性能？

- 减少磁盘 I/O 操作：缓冲区扩大后可存储更多数据，减少从磁盘读取的频率，降低 I/O 开销。
- 提高数据访问速度：内存访问速度远快于磁盘，更多数据存储在缓冲区中能加快读取速度。
- 提升缓存命中率：缓冲区容量增大，提高了查询数据直接从内存获取的可能性。
- 优化事务处理：事务过程中数据可保存在缓冲区，减少频繁磁盘写入，提高事务效率。
- 增强并发性能：在高并发场景下，缓冲区存储更多数据，减少查询对磁盘资源的争用。
- 支持复杂查询：缓冲区扩大可在内存中保存中间结果和索引数据，提升复杂查询性能。
- 减少写操作延迟：缓冲区扩大后，写入操作可以批量处理，降低磁盘写入频率。
- 缓存更多索引：较大的缓冲区可存储更多索引页，加速索引查找操作。

6. 堆文件、直接文件和索引文件各适用于何种场合？

堆文件：

- 适用场合：数据插入频繁但查询不常用的场景，例如日志记录或历史数据存储。

- 特点：数据无特定顺序存储，插入效率高，但查找和更新效率低。

直接文件：

- 适用场合：需要通过唯一标识快速定位数据的场景，例如用户账号信息、商品库存管理。
- 特点：基于哈希算法直接定位数据，查找速度快，但不适合范围查询。

索引文件：

- 适用场合：需要频繁查询且要求快速访问的场景，例如数据库中支持主键或多列条件查询的表。
- 特点：数据按索引结构组织，适合范围查询和复杂条件查找，但插入和删除的开销较大。

7. 为什么主键上一般都建有主索引？

- 唯一性约束：主键必须唯一，主索引能够快速验证数据的唯一性，保证数据完整性。
- 高效查询：主键常用于定位单条记录，主索引通过索引结构（如 B+树）提供高效的查找能力。
- 数据排序：主索引将主键数据按顺序组织，便于范围查询和排序操作。
- 关联查询优化：主键通常用于外键关联查询，主索引提高了连接操作的性能。
- 数据存储优化：主索引有助于组织数据存储，使数据检索更加高效。
- 系统默认行为：许多数据库系统默认在主键上创建主索引以支持高效的主键操作。

8. 当查询的元组数在总元组数中所占的比重较大时，为什么用非簇集索引有时反而不利？为什么在此情况下用簇集索引有利？

非簇集索引不利的原因：

- 数据物理存储无序：非簇集索引的指针可能随机指向磁盘不同位置，导致大量磁盘 I/O 操作。
- 扫描效率低：查询元组占比大时，需要频繁跳转读取多个数据块，性能下降。

簇集索引有利的原因：

- 数据物理存储有序：簇集索引按关键字顺序存储数据，相关元组集中存放，减少磁盘 I/O 操作。
- 顺序读取高效：查询涉及的大量元组可以通过顺序扫描磁盘实现，读取效率更高。
- 适合范围查询：簇集索引优化了范围查询的性能，尤其是涉及较多数据时更为高效。

9. 在数据库系统中，散列一般用的不如索引多，试分析其原因。

- 不支持范围查询：散列结构只适合精确查找，无法高效处理范围查询，而索引（如 B+树）能支持高效的范围查询。
- 冲突处理复杂：散列结构可能发生哈希冲突，处理冲突需要额外开销，影响性能和维护复杂度。
- 动态调整困难：数据量变化时，散列结构需要重新调整或重建哈希表，而索引结构（如 B+树）能动态平衡，适应性更强。
- 不支持排序：散列无法提供有序数据访问功能，而索引结构能高效支持排序操作。
- 多条件查询不便：散列结构通常针对单一属性设计，难以支持多列组合查询，而索引支持多列或联合索引。
- 适用场景有限：散列主要适用于唯一键查找和精确匹配查询，场景较为局限，而索引适用于更广泛的查询需求。
- 维护复杂性高：散列在插入、删除时可能需要复杂的冲突解决或重哈希操作，维护成本高于索引。

10. 如何减少 B-树的级数？

- 增大页节点的存储容量：通过增大每个 B-树节点的页大小，能存储更多关键字，从而减少树的高度。
- 使用高效存储结构：选择压缩或紧凑的存储方式，减少关键字存储占用的空间，容纳更多关键字。
- 合理选择数据类型：采用占用空间较小的数据类型（如整数替代字符串）存储关键字，增加每个节点的存储数量。
- 减少插入和分裂：优化数据的插入顺序，尽量避免节点分裂，减少级数增加的可能性。
- 预分配空间：在设计表时根据数据量预估索引大小，优化树的构建以降低级数。

11. B 树和 B+树有什么区别？为什么在 DBMS 中使用 B+树？（P107）

在 B+树中，键是存储在内部节点中的索引，记录存储在叶子节点中。在 B 树中，键不能重复存储，这意味着没有重复的键或记录。在 B+树中，叶子节点相互链接以提供顺序访问。在 B 树中，叶节点彼此不链接。

B+树是一种平衡的叉搜索树。B+树确保所有叶子节点保持相同的高度。在 B+树中，叶节点使用链表链接。因此，B+树可以支持随机访问和顺序访问。

12. B-树为代表的树形索引成为当前数据库系统主流索引是否具有必然性？

索引文件是一种适应面比较广的文件结构，因此在数据库系统中得到了广泛的应用。对于经常变动的文件，静态索引的性能会随时间变化而变坏，所以目前在数据库系统中应用更多的是动态索引。而 B 树就是为了磁盘或其它存储设备而设计的一种平衡多分树，能很好地进行动态索引。B 树提供了三种存取路径：

1. 通过索引集进行树形搜索；
2. 通过顺序集进行顺序搜索；
3. 先通过索引找到入口，再沿顺序集顺序搜索。

B 树不仅提供了灵活的存取路径，而且能够自动保持平衡，不须定期重组，因此 B 树为代表的树索引系列在数据库系统中应用甚广，成为数据库主流具有必然性。

13. 计算 B+树的秩 k

假设物理块的有效大小 $B=492$ 字节，块的指针为 6 字节，Sailors 表的 sid 属性 2 字节，sname 占 4 字节，rating 占 1 字节，age 占 1 字节，master 占 2 字节。每个记录除属性外，还需增加一个字节作为删除标记。

(1) 若 Sailors 表的 rating 属性上建有 B+树簇集索引，求 B+树的秩 k？

索引集： $2k * 1 + (2k+1) * 6 \leq 492$

顺序集： $2k * 1 + 2 * 6 + 2k * 6 \leq 492$

(2) 若 Sailors 表的 sid 属性上建有 B+树的主索引，求 B+树的秩 k？

索引集： $2k * 2 + (2k+1) * 6 \leq 492$

顺序集： $2k * 2 + 2 * 6 + 2k * (6+2) \leq 492$

第六章 查询处理优化

1. 从查询优化的角度解释为什么在 SQL 的 WHERE 子句中应该避免使用“OR”（P124）

SQL 查询优化——OR 与 UNION

- OR 需要第 2 阶段处理
- 考虑将查询重写为 2 个 SELECT 的并集，使索引访问成为可能
- UNION ALL 避免排序，但包含重复项
- 监控并解释查询，以确定哪个最好

为了加快查询速度，优化查询效率，主要原则就是应尽量避免全表扫描，尽量在 where 及 order by 涉及的列上建立索引。然而在 where 子句中使用 or 来当连接条件时，会导致引擎放弃使用索引而进行全表扫描，改用 Union 后，性能会大大提高。

使用 Or 谓词的析取选择条件，并没有好的优化条件，只能按其中各个条件分别选出一个元组集，再求这些元组集的并。并是开销大的操作，而且在 OR 连接的多个条件中，只要有一个条件没有合适的存取路径，就不得不采用顺序扫描来处理这种查询，导致效率大大降低。

2. 数据模型的优化包含哪些方法？

- 1、确定各属性间的函数依赖关系。根据需要分析阶段得出的语义，分别写出每个关系模式的各属性之间的函数依赖以及不同关系模式中各属性之间的数据依赖关系。
- 2、对各个关系模式之间的数据依赖进行极小化处理，消除冗余的联系。
- 3、判断每个关系模式的范式，根据实际需要确定最合适的范式。
- 4、根据需要分析阶段得到的处理要求，分析这些模式对于这样的应用环境是否合适，确定是否要对某些模式进行分解或合并。

3. Join 的优化

join的优化:

- buffer: 两个缓冲，大小是物理块大小的整数倍，一个给外循环表，一个给内循环表。外循环中，一次取好多物理块，内循环也是如此；每次比较时，两个缓冲的内容两两比较。
 - 分配缓冲区，希望读物理块个数最少：外循环n-1，内循环1，内循环遍历次数少。
- merge scan: 需要两个关系 R S，在磁盘上【按照连接属性的值】，事先做好外排序。双指针法，每个表扫一次。
- B+树索引/哈希索引：
 - 把【连接属性上有索引】的表作为内循环表。根据外循环缓冲中的值，查内循环表的索引，再也不用扫描内循环表了。
 - 如果匹配tuple的数量达到内循环表的20%，用（非簇集的）索引不合算。

4. 单纯代数优化的局限性

- 缺乏代价信息：单纯代数优化只基于等价变换规则，无法评估查询执行的实际代价，可能导致低效执行计划。
- 无法考虑物理实现：代数优化忽略了底层存储结构（如索引、分区）和访问方法（如顺序扫描、索引扫描）的影响。
- 不支持选择性评估：无法根据条件的选择性来调整查询顺序或减少不必要的中间结果。
- 忽略系统资源：不考虑内存、CPU、I/O 等系统资源的使用情况，可能导致资源消耗不均衡。
- 难以处理复杂查询：对于复杂的多表连接或嵌套查询，单纯代数优化可能无法生成高效的执行计划。
- 固定转换规则：基于规则的优化缺乏灵活性，可能错过更优的查询计划。

第七章 事务管理

1. 数据冗余的优缺点是什么？

数据冗余会造成存储浪费，造成数据不一致，带来版本控制问题。

数据冗余的优点有：对数据库恢复，数据冗余是必须的。

2. 数据库恢复的基本原则是什么？具体实现方法是什么？（P138）

对于恢复，数据冗余是必须的。

具体的恢复技术大致为下列三种：

（1）单纯以后备副本为基础的恢复技术：核心是周期性地把磁盘上的数据库转储(dump)到磁带上，这样的数据库副本可以脱机存放，不受系统故障影像，成为后备副本。

（2）以后备副本和运行记录为基础的恢复技术：通过运行记录保持恢复所必需的数据，运行记录一般不能和数据库放在同一磁盘上，因经常把运行记录复制到磁带上，防止磁盘故障导致数据“全军覆没”。

(3) 基于多副本的恢复技术：设置多个具有独立失效模式的数据库副本，这些副本可以互为备份，用于恢复。

3. 列出 DBMS 三种恢复策略，并描述提交规则和提前记录规则对于更新事务的必要性。

1. 后像在事务提交前完全写入数据库
2. 后像在事务提交后才写入数据库
3. 后像在事务提交前后写入数据库

必要性：

提交规则是保证后像在事务提交前写入非易失存储器，这样即使事务进入提交阶段以后发生故障，仍可利用记录下来的后像重做更新，从而确保事务满足 ACID 原则；

先记后写规则是指如果后像在事务提交前直接写入数据库，则必须在此之前将相应的前像写入运行记录(Log)，以备当事务进入提交阶段之前发生故障时做 undo,使事务的执行满足 ACID 原则。

4. 失效及恢复对策 (P146)

假设运行记录与数据库的存储磁盘有独立失效模式，介质失效恢复时，对运行记录中上一检查点以前的已提交事务应该 redo 否？为什么？什么时候可以清空 CTL 表？

介质失效指磁盘发生故障，数据库受损，如磁盘、刺头破损。

介质失效后应该在新介质中加载最近后备副本，并用档案存储器内运行记录中的后像，redo 后备副本以后提交的所有更新事物。如果检查点比后备副本要新，则对后备副本以后，检查点以前的事物也应该 redo。

在取后备副本后，以前的运行记录就失去价值，可以清空 CTL 表。

5. 登记日志文件时为什么必须先写日志文件，后写数据库？

把数据的修改写到数据库和把对数据的修改操作写到日志文件是两个不同的操作，在两个操作之间可能会发生故障如果先写了数据库修改，而在日志文件中没有登记这个修改，在恢复的时候就无法恢复这个修改了；如果先写日志，但没有修改数据库，按日志恢复时只是多执行一次撤消操作，并不影响数据库的正确性，为了安全，一定要先写日志文件。

6. 试述使用检查点方法进行恢复的步骤

- 1) 从重新开始文件中找到追后一个检查点记录在日志文件中的地址，由该地址在体制文件中找到最后一个检查点记录。
- 2) 由该检查点记录得到检查点建立时刻所有正在执行的事务清单 ACTIVE-LIST。这里建立两个事务队列：
 - ①UNDO-LIST：需要执行 undo 操作的事务集合；
 - ②REDO-LIST 需要执行 redo 操作的事务集合。把 ACTIVE-LIST 暂时放入 UNDO-LIST 队列，REDO 队列暂时为空。
- 3) 检查点开始正向扫描日志文件：
 - ①如果有新开始的事务 T_i ，把 T_i 暂时放入 UNDO-LIST 队列；
 - ②如有提交的事务 T_j ，把 T_j 从 UNDO-LIST 队列移到 REDO-LIST 队列，直到日志文件结束；
- 4) 对 UNDO-LIST 中的每个事务执行 UNDO 操作，对 REDO-LIST 中的每个事务执行 REDO 操作。

7. Checkpoint

- (1) Checkpoint 用于数据库发生系统失效时，避免大量的无效 redo。
- (2) 可以减少，具体与取 CP 的周期有关。
- (3) 取后备副本后可以清空。

8. 试叙述事务的 4 个性质，并结合每一个性质由 DBMS 的哪个子系统实现？每一个性质对 DBS 有什么益处？

1) 事务的原子性是指一个事务对 DB 的所以操作，是一个不可分割的工作单位。原子性是由 DBMS 的事务管理子系统实现的。事务的原子性保证了 DBS 的完整性。

2) 事务的一致性是指数据不会因事务的执行而遭受破坏。事务的一致性是由 DBMS 的完整性子系统实现的。事务的一致性保证数据库的完整性。

3) 事务的隔离性是指事务的并发执行与这些事务单独执行时结果一样。事务的隔离性是由 DBMS 的并发控制子系统实现的。隔离性使并发执行的事务不必关心其他事务，如同在单用户环境下执行一样。

4) 事务的持久性，是指事务对 DB 的更新应永久地反映在 DB 中。持久性是由 DBMS 的恢复管理子系统实现的。持久性能保证 DB 具有可恢复性。

9. 什么是数据库镜像？它有什么用途？

根据 DBA 的要求，自动把整个数据库或其中的关键数据复制到另一个磁盘上。每当主数据库更新时，DBMS 自动把更新后的数据恢复过去，即 DBMS 自动保证镜像数据与主数据的一致性。用途：

1) 一旦出现介质故障，可由镜像磁盘继续提供使用，同时 DBMS 自动利用镜像磁盘数据进行数据库恢复，不需要关闭系统和重装数据库副本。

2) 在没有出现故障时，数据库镜像还可以并发操作。提高数据库的可用性。

10. 在数据库中为什么要有并发控制机制？

数据库系统一个明显的特点是多个用户共享数据库资源，尤其是多个用户可以同时存取相同数据。在这样的系统中，在同一时刻同时运行的事务可达数百个。若对多用户的并发操作不加控制，就会造成数据库存、取的错误，破坏数据的一致性和完整性。所以数据库中要有并发控制机制。

11. 锁 (P153)

11.1 如果计息业务设计为单独计算单个账户的利息，即 T2 (a) 计算账户 a 的利息，T2 (B) 计算账户 B 的利息。

这个方案正确吗？为什么？

不正确，不符合一致性原则。对于两个账户增加利息后的总资产值，应在一个事务内计算。如果分开计算，在先计算了 A 的利息后，如果 A 对 B 进行了一次转账，再计算 B 的利息会导致的利息不精确，出现类似读脏数据的问题。

11.2 加锁协议的基本思想是什么？它如何确保正确执行并发事务？采用锁定协议后必须解决的问题是什么？

基本思想：并发事务在对同一个数据对象进行操作之前，会向系统发送一个锁定操作对象的请求。在事务的锁请求被批准后，它才能对对象进行一定的控制。

由于在该事务释放锁之前，其他事务无法获取数据对象的锁请求并对其进行操作，从而避免了访问冲突，保证了并发事务的正确执行。

采用锁定协议后，可能会出现活锁、死锁等问题，其中必须解决的问题是事务之间的循环等待造成的死锁问题。

11.3 已有的 (S,X)、(S,U,X) 锁能解决事务并发中的死锁问题么？为什么？

不能解决并发事务中的死锁问题。当一个事物 A 占用数据对象 a 的 X 锁，事务 B 占用数据对象 b 的 X 锁，事务 A 和事务 B 又分别申请数据对象 b 和数据对象 a 的锁，在 (S, X) 和 (S, U, X) 锁中，均无法获准，需要等待对方事务释放锁，而进入等待状态则无法释放自己所占用的锁，从而陷入循环等待，即死锁。

11.4 (S,U,X)锁的相容矩阵，为什么已经加了 U 锁，不允许其它事务申请加 U 锁？ 如果允许会出现什么情况？

U 锁表示事务对数据对象进行更新的操作，但事实上在最后写入阶段事务需要再将其升级为 X 锁才行。如果此时允许其他事务再申请 U 锁，导致最终写操作时无法申请到 X 锁而导致死锁。比如事务 A 想将 U 锁升级为 X 锁进行数据写操作时，由于存在其他事物对数据对象的 U 锁，而无法升级为 X 锁，从而导致死锁。

11.5 分别讨论写锁和读锁在事务结束前释放可能引起的问题

- 写锁在事务结束前释放可能引起的问题：
 - 丢失更新：如果写锁提前释放，其他事务可能修改相同的数据，导致原事务的更新被覆盖。
 - 数据不一致：写锁提前释放可能导致其他事务读取到未完成事务的中间状态，破坏数据一致性。
 - 脏写问题：一个事务在未提交数据时，其他事务可能修改该数据，导致后续回滚操作引发混乱。
- 读锁在事务结束前释放可能引起的问题：
 - 不可重复读：读锁提前释放，其他事务可能修改数据，导致同一事务多次读取同一数据结果不一致。
 - 幻读问题：读锁提前释放，其他事务可能插入或删除数据，导致范围查询的结果不一致。
 - 数据不一致：多个事务并发读取和修改同一数据，可能导致不一致的查询结果。

11.6 什么叫活锁？如何防止活锁？

- 什么叫活锁：
 - 活锁是指某个事务或进程由于总被其他事务或进程抢占资源而无法继续执行的情况。
 - 活锁通常发生在事务不断重试但无法获得资源的场景，表现为事务一直活着但无法前进。
- 如何防止活锁：
 - 使用公平的调度策略：采用先来先服务（FIFO）策略，按事务或进程的到达顺序分配资源。
 - 设置重试次数限制：限制事务重试的次数，超出限制后采取其他措施（如回滚或暂停）。
 - 引入随机等待：在冲突时引入随机等待时间，减少资源争用时的冲突概率。
 - 提高优先级：对长期等待的事务提高优先级，使其优先获得资源。
 - 使用死锁检测机制：通过定期检测等待链，避免长时间资源争用导致活锁。

11.7 什么叫死锁？如何对付死锁？如何处理死锁？

- 什么叫死锁：
 - 死锁是指两个或多个事务相互等待对方释放资源，导致无法继续执行的情况。
 - 事务之间形成了循环等待关系，所有相关事务都无法完成。
- 如何对付死锁：
 - 避免死锁：通过设计策略防止死锁的发生，如资源按固定顺序申请，避免循环等待。
 - 检测死锁：定期检查事务等待图，如果发现循环依赖则确认发生死锁。
 - 预防死锁：使用如资源有序分配、单一资源分配的预防机制，避免死锁条件的形成。
- 如何处理死锁：
 - 超时机制：为事务设置超时时间，超时后强制回滚，解除死锁。
 - 死锁回退：选择一个事务（如最少资源占用的事务）回滚，释放资源以解除死锁。
 - 死锁检测与恢复：检测死锁后，通过中止一个或多个事务打破循环依赖。
 - 乐观并发控制：减少锁的使用，从机制上降低死锁发生的概率。

11.8 试区别串行调度与可串行化调度，请各举一例

- 串行调度：
 - 定义：事务按顺序依次执行，一个事务完成后另一个事务才开始执行，完全没有交错操作。
 - 特点：绝对不会发生并发冲突，事务结果总是正确的。
 - 例子：事务 T1 更新账户 A 余额，然后事务 T2 查询账户 A 余额。T1 执行完后，T2 才开始执行。
- 可串行化调度：
 - 定义：多个事务的操作交错执行，但其最终执行结果等价于某种串行调度的结果。

- 特点：允许并发操作，提高性能，同时保持事务的一致性和正确性。
- 例子：事务 T1 更新账户 A 余额，事务 T2 读取账户 A 余额。如果调度确保 T2 读取的是 T1 更新后的余额，则结果等价于 T1 先执行完再执行 T2。
- 区别：
 - 串行调度是事务严格按顺序执行，而可串行化调度允许并发，但结果必须等价于某种串行执行的结果。
 - 串行调度完全避免了并发冲突，可串行化调度通过机制保证并发执行的正确性。

12. 各种数据有不同的恢复要求：有些要求恢复至最近一致状态；有些只要求恢复至先前的某个一致状态；也有些数据不需要恢复措施，只需在失效后重建，试各举一例

- 银行交易系统的账户余额
银行交易系统需要将数据恢复到最近一次一致状态，以确保所有已提交的交易（如存款、取款）都正确反映在账户余额中，而未提交的交易被回滚。
- 在线购物网站的订单历史
在系统出现故障时，只需将订单记录恢复到上一次定期备份的一致状态，而不需要恢复到系统故障前的状态，因为某些未处理的订单可以重新生成。
- 搜索引擎的索引文件
搜索引擎的索引数据是从原始数据生成的缓存文件，如果索引失效或损坏，可以直接通过重新扫描原始数据来重建，无需进行恢复。

13. 试论述提交规则和先记后写规则对更新事务的必要性

提交规则的必要性：

- 确保事务的原子性：提交规则规定只有当事务完成所有操作后，才能将事务结果持久化，保证事务的完整性。
- 防止未完成事务对数据的影响：未提交的事务结果不能被其他事务读取，防止不一致数据的传播。
- 支持系统恢复：提交点明确区分已完成事务和未完成事务，有助于系统在故障后进行正确的 redo 和 undo 操作。

先记后写规则的必要性：

- 确保数据恢复的一致性：事务修改的数据必须先记录到日志中，防止在系统崩溃后丢失未记录的修改。
- 提高系统可靠性：日志中包含了更新前后的数据状态，支持事务的回滚（undo）和重做（redo），保障数据一致性。
- 支持持久性：先写日志确保在事务提交后，即使系统故障，也可以通过日志恢复事务的所有修改。
- 避免写入失序：日志先于数据写入，确保在故障发生时，系统能够通过日志完整回溯事务操作过程。

14. 说明 redo、undo 操作内容及其所需的条件，在什么情况下需要 redo，在什么情况下需要 undo？

- undo 操作内容：

- 回滚未提交事务的所有修改，将数据恢复到事务开始前的状态。
- 基于日志中的“前像”（before image）还原数据。

- undo 所需条件：

- 系统中存在未提交的事务。
- 日志中记录了未提交事务的修改操作及其前像。

- redo 操作内容：

- 重做已提交事务的所有修改，将数据恢复到事务提交后的状态。

- 基于日志中的“后像”（after image）重新应用修改。
- **redo 所需条件：**
 - 系统发生故障，但有事务已成功提交。
 - 日志中完整记录了已提交事务的修改操作及其后像。
- **需要 redo 的情况：**
 - 数据库发生故障后，部分已提交事务的修改尚未写入物理数据库，需要通过 redo 操作重新应用已提交的修改。
- **需要 undo 的情况：**
 - 数据库发生故障后，存在未提交事务，为保证一致性，需要通过 undo 操作回滚未完成的事务。

15. 讨论异地更新恢复技术的优缺点。在异地更新方案中，是否需要运行记录？在什么环境下可以省去运行记录？

- **异地更新恢复技术优点：**
 - 提高数据可靠性：在发生灾难或故障时，异地备份可以恢复数据，保障业务连续性。
 - 实现容灾：通过异地存储与恢复，减少数据丢失风险，适用于灾难恢复场景。
 - 提高数据访问性能：在某些情况下，异地节点可以分担读操作负载，提高整体性能。
- **异地更新恢复技术缺点：**
 - 成本高：需要额外的硬件、存储和网络资源支持异地备份与恢复。
 - 网络延迟：数据同步到异地节点需要时间，可能影响实时性。
 - 数据一致性难保证：多节点同步更新时可能导致数据不一致，需要额外的协调机制。
 - 恢复时间较长：异地恢复可能需要传输大量数据，延长恢复时间。
- **是否需要运行记录：**
 - 在异地更新方案中，一般需要运行记录（日志）以追踪事务更新并确保数据恢复的完整性和一致性。日志能够帮助执行 undo 和 redo 操作，支持异地节点同步恢复。
- **在什么环境下可以省去运行记录：**
 - 异地节点实时同步：如果异地节点始终实时同步主节点的所有数据，并保证同步过程中的数据完整性，则可以省去运行记录。
 - 数据可完全重建：如果数据可以通过重新计算或重新生成获得（如索引文件或缓存数据），则不需要运行记录。
 - 数据不需要强一致性：对于某些对一致性要求较低的场景（如只读备份），可以不使用运行记录。

16. 什么叫并发？为什么要并发运行？并发会引起什么问题？什么样的并发执行才是正确的？如何避免并发所引起的问题？

- **什么叫并发：**
 - 并发是指多个事务或任务在同一时间段内交替执行，共享系统资源的运行方式。
- **为什么要并发运行：**
 - 提高系统资源利用率：并发运行使 CPU、内存和 I/O 等资源得以高效利用。
 - 提升系统吞吐量：支持多个用户或事务同时处理，提高数据库处理能力。
 - 减少用户等待时间：并发运行允许多任务并行，改善用户体验。
- **并发会引起什么问题：**
 - 脏读：一个事务读取了另一个未提交事务的修改数据。
 - 不可重复读：一个事务两次读取同一数据时，值发生了变化。
 - 幻读：一个事务读取范围数据时，由于其他事务插入或删除数据，导致结果不一致。
 - 丢失更新：多个事务同时更新同一数据，后一个覆盖了前一个的更新。
- **什么样的并发执行才是正确的：**

- 满足 ACID 原则：并发执行必须保证事务的原子性、一致性、隔离性和持久性。
- 遵循可序列化调度：并发事务的执行结果与按某一顺序串行执行的结果相同。
- 如何避免并发所引起的问题：
 - 使用锁机制：通过共享锁和排他锁控制事务对数据的并发访问。
 - 事务隔离级别：设置如读未提交、读已提交、可重复读、可序列化等隔离级别以减少并发冲突。
 - 乐观并发控制：在提交时检查冲突，若有冲突则回滚重试。
 - 悲观并发控制：在操作前锁定资源，防止其他事务并发修改。
 - 时间戳机制：为每个事务分配时间戳，确保事务按时间顺序执行。

(6) 既然目标可串行化调度比冲突可串行化调度多，我们为什么要强调冲突可串行化而非目标可串行化呢？

- (1) 是指 DBMS 可同时接纳多个事务，事务在时间上可以重叠执行；
- (2) 目的：①改善系统资源利用率；②改进响应时间；
- (3) 问题：①丢失更新；②读脏数据；③读值不可重复；
- (4) 可串行化；
- (5) 可采用封锁法、时间戳法等并发控制方法；
- (6) 目标可串行化的判断算法是 NP 完全问题，也没有保证目标可串行化的简单实用的规则；冲突可串行化覆盖了绝大部分可串行化的调度实例，测试算法简单、易实现。

第八章 数据库安全和完整性约束

1. 什么是数据库的完整性？它和数据库的安全性有什么不同？

数据库的安全性是针对“人为的破坏，比如数据被不该知道的人访问、篡改甚至破坏”的情况；数据库的完整性是针对“输入或者更新数据库的数据有误，更新事务为遵从保持数据库一致性原则”的情况。

2. 关系模式的设计考虑哪些问题？

考虑了完整性约束和规范化的问题。

- ①数据库中各个关系表都有主键，主键的值是唯一的且不为空，满足了实体完整性约束。同时 enroll 表中定义了对 course 和 student 表的外键，满足了引用完整性约束。
- ②关系中的属性都是原子的，因此满足第一范式的要求；同时各关系表中不存在部分函数依赖和传递函数依赖，因此该数据库的设计也满足第二和第三范式的要求。

第九章 触发器和主动数据库系统

1. 解释什么是触发器并列出发触器的组件

触发器：当 DBMS 发生指定更改时自动启动的过程

三个部分：

事件（激活触发器）

条件（测试触发器是否应运行）

操作（如果触发器运行会发生什么）

2. 试编写一个触发器 (P185)

监视上面数据库中 enroll 表上的 INSERT 操作, 对每条 INSERT 语句, 判断其插入元组的 grade 值是否小于 3.0, 将这样的元组自动插入到 failedcourse 表中(设 failedcourse 表与 enroll 表的模式完全相同)。

```
CREATE TRIGGER insert_grade_check
AFTER INSERT ON enroll
REFERENCING NEW TABLE AS NE
FOR EACH STATEMENT
WHEN (EXISTS(SELECT * FROM NE WHERE grade<3.0))
INSERT INTO failedcourse
    SELECT *
    FROM NE
    WHERE NE.grade<3.0
```

第十章 数据依赖和关系模式的规范化

1. 是否存在“删除异常”和“修改异常”问题? 如果存在, 请说明分别是怎样发生的?

(P198)

存在。

“删除异常”：发生在当删除某门课程信息的时候会导致教师信息也被删除而丢失的情况。

“修改异常”：发生在当一个教师离开原本部门去到其他部门时, 需要修改该老师的所有记录。

2. 数据模式遵循的范式越高越好吗?

数据库的范式主要目的是防止数据冗余、更新异常、插入异常和删除异常, 因此, 如果达到了该目的也就可以了, 但范式越高可能带来处理速度缓慢和处理逻辑复杂的问题, 因此需要权衡考虑。并不是应用的范式越高越好, 要看实际情况而定。应用的范式等级越高, 则表越多。表多会带来很多问题: 1 查询时要连接多个表, 增加了查询的复杂度 2 查询时需要连接多个表, 降低了数据库查询性能

3. 范式

1NF: atomic.

2NF: 1NF, 且不存在部分函数依赖 (只依赖一个主键)。

3NF 【常用】: 2NF, 且不存在属性对主键的传递依赖。

4NF: 3NF, 无多值依赖。5NF: 4NF, 无连接依赖。

insert(难以记录信息) / delete abnormality(删没了就丢失信息), redundancy / hard to update.

criterion: 一事一地, one fact in one place。

SQL

- 结尾的 “;” 记得加
- 查找所有 xxx，可以查出所有 xxx 数量，然后进行数量匹配，要 distinct xxx，如 distinct cno
Eg. 查找学习所有课程的学生姓名

```
SELECT sname
FROM S
WHERE sno IN (
    SELECT sno
    FROM SC
    GROUP BY sno
    HAVING COUNT(DISTINCT cno) = (
        SELECT COUNT(*)
        FROM C
    )
);
```

- 查找所有 xxx，反向思维，查找一个 A，不存在一个 B 没有被 A 怎么怎么
Eg. 查找学习所有课程的学生姓名——查找一个学生，不存在一门课程没有被学生学过。

```
SELECT sname
FROM S
WHERE NOT EXISTS (
    SELECT *
    FROM C
    WHERE NOT EXISTS (
        SELECT *
        FROM SC
        WHERE SC.Sno = S.Sno
        AND SC.cno = C.cno
    )
);
```

- in{a, b}
- 嵌套关联查询：exists
- 嵌套非关联查询：in
- 带有 EXISTS 谓词的子查询不返回任何实际数据，它只产生逻辑值
- 字符匹配：
姓刘：name like ‘刘%’
姓刘且名字为两个字：name like ‘刘_’
- 排序：
先对 A 升序，再对 B 降序：ORDER BY A ASC, B DESC
- Where 中不可对同一表同一属性筛选不同条件，会导致总是查询结果为空
Eg. Where SC.CNO=’S2’ and SC.CNO=’S4’不正确

- 一个表可以重命名为两个，表示自连接，可用于筛选 xxx and xxx

Eg.筛选至少选修课程号为 C2 和 C4 的学生学号

```
SELECT sno
FROM sc as X, Y
WHERE X.sno = Y.sno
      AND X.cno = 'C2'
      AND Y.cno = 'C4';
```

(5) 按医生职称查询 2019 年各职称级别完成手术次数最多的医生姓名及其完成的手术次数。(8 分)

```
SELECT 职称, 姓名, 手术次数
FROM 医生, (
    SELECT 医生编号, COUNT (*) AS 手术次数
    FROM 医生, 手术医生安排 AS A
    WHERE 手术日期.GetYear () = 2019 and 医生.医生编号=手术医生安排.医生编号
    and NOT EXISTS
    (SELECT * FROM 医生, 手术医生安排 AS B
    WHERE 医生.医生编号=手术医生安排.医生编号 and 手术日期.GetYear ()
    =2019 and COUNT(B.医生编号) > COUNT(A.医生编号) and A.职称=B.职称
    GROUP BY 医生编号)
    GROUP BY 医生编号
} AS S
WHERE 医生.医生编号=S.医生编号;
```

查询 Computer Sci 系所开设各门课程的最高分及取得最高分学生的学号

```
SELECT E.grade, E.sid FROM enroll E,
(SELECT MAX(E1.grade) AS maxgrade, E1.cno FROM enroll E1
WHERE E1.dname1='Computer Sci' GROUP BY E1.cno) AS TEMP
WHERE E.dname1='Computer Sci'
AND E.cno=TEMP.cno AND E.grade=TEMP.maxgrade
```

查询所学每一门课程成绩均高于等于该课程平均成绩的学生的姓名及 相应课程号

- 使用 不存在 小于来写较为容易

```
SELECT SNAME, CNO
FROM STUDENT, SC
WHERE STUDENT.SNO = SC.SNO AND STUDENT.SNO NOT IN(
    SELECT SNO FROM SC SCX
    WHERE SCX.GRADE < (SELECT AVG(GRADE)
        FROM SC SCY
        WHERE SCY.CNO = SCX.CNO
    )
);
```